

joaogoncalves.net

Angular Avançado

Sessão 1 · Arranque, anatomia de um projeto Angular 22 e componentes baseados em Signals

moOngy · 8 set – 22 out 2026 · 14 sessões · MS Teams

João Gonçalves · joaogoncalves.net





Quem vos acompanha

João Gonçalves

- Interactive Developer / Trainer / Consultant
- Licenciatura em Matemática Aplicada · Mestrado em Comunicação Multimédia
- Formação e consultoria em front-end há décadas
- Angular desde o AngularJS — e a acompanhar cada reinvenção do framework

Contactos

joaogoncalves.net
edu@joaogoncalves.net
@joaopapin

“Making the complicated simple.”



E vocês?

1 Nome e equipa

Em que projeto estão atualmente?

2 Versão de Angular

Qual a versão em produção? Ainda há NgModules?

3 Signals

Já usam signal(), computed(), input()? Ou só RxJS?

4 Uma expectativa

O que gostavam de levar daqui no final das 14 sessões?



Onde vive o material

Repositório do projeto integrador

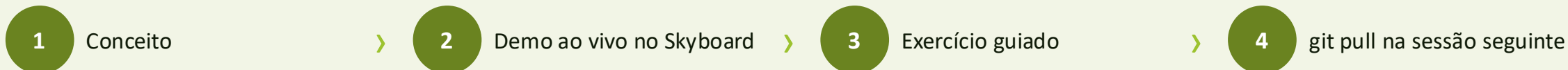
github.com/JoaoGoncalves/skyboard

Público. Código de cada sessão fica etiquetado (sessao-01, sessao-02, ...). No início de cada sessão: git pull.

Comunicação

Chat do canal da formação no MS Teams para links, dúvidas entre sessões e partilha de snippets.

Como vamos trabalhar





O percurso: 8 módulos em 14 sessões

01

Evolução e arquitetura moderna

standalone-first · CLI · feature-first

02

Templates, control flow e rendering

@if / @for / @switch / @let · @defer

03

Routing avançado e DI

provideRouter · guards funcionais · inject()

04

Componentes modernos

input() · output() · model() · composição

05

Signals e RxJS

computed · effect · interop · critérios

06

HTTP moderno

provideHttpClient · interceptors · httpResource

07

Estado e forms

stores · Typed Forms · Signal Forms

08

Zoneless e performance

Zone.js · SSR/hydration · DevTools

Hoje tocamos nos módulos 01, 02 e 04 — o resto vai sendo aprofundado ao longo das sessões.



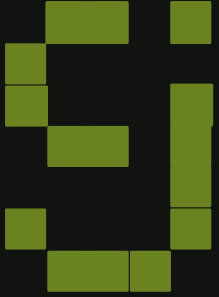
Skyboard: o projeto que vamos construir



- **Domínio:** pesquisa e seleção de voos — dados simples, mas suficientes para HTTP, estado, forms e routing.
- **Hoje:** pesquisa de voos (from/to), lista de resultados, seleção e um primeiro carrinho.
- **Depois:** detalhe do voo, edição com Signal Forms, store compartilhado, lazy loading, zoneless.
- **Stack:** Angular 22 · TypeScript 6 · Node LTS · sem Angular Material no arranque.

```
estrutura feature-first

src/app/
├── shell/           # navbar, sidebar
├── flight-search/  # feature
│   └── data/       # flight, aircraft, price
├── shared/
│   ├── ui/         # reutilizáveis
│   └── util/       # helpers
├── app.ts / app.html
├── app.config.ts
└── app.routes.ts
```



PARTE 1

Anatomia de um projeto Angular 22

Tooling · ng new · estrutura · bootstrap · configuração do CLI



Ambiente de trabalho

Node.js LTS

Mínimo 22.22.3 (linha 22) ou 24.15.0 (linha 24). Recomendado: 24 LTS. Versões ímpares (23, 25) não são suportadas. Inclui o npm — mínimo 8, recomendado 12 (vem com o Node 24). Vários projetos → nvm.

Angular CLI

`npm install -g @angular/cli@22` — mesma major do `@angular/core` (22.x). Verificar com `ng version`. Gerar, servir e compilar.

VS Code + Angular Essentials

Language Service (autocomplete em templates), ESLint, debugger. Abrir sempre a pasta raiz (a que tem `angular.json`).

Git

Para clonar o Skyboard e acompanhar as sessões. O TypeScript (~6.0) vem com o projeto — não instalar globalmente.



Do zero ao browser em dois comandos

terminal

```
ng new skyboard          # aceitar defaults (Enter)
cd skyboard
ng serve -o              # http://localhost:4200
ng serve -o --port 4242
```

- **ng new** descarrega e configura compilador TS, build (esbuild/Vite) e estrutura de pastas.
- **ng serve** dev server + watch: recompila e atualiza o browser a cada gravação.
- **Quando dessincroniza** (vários ficheiros gravados de seguida, renames): gravar de novo ou reiniciar o ng serve.

experiência

```
// src/app/app.ts
export class App {
  protected readonly title = signal('skyboard');
}
```

Mudar 'skyboard' → 'World!' e gravar: o browser atualiza sozinho. Primeiro contacto com um signal — voltamos já a ele.

Instalar mais pacotes

```
npm i <pkg> — só instala
ng add <pkg> — instala e configura (schematics), ex.: ng add
@angular/material
```



Os ficheiros que interessam

<code>app.ts</code>	Classe do componente raiz — comportamento.
<code>app.html</code>	Template do componente raiz — aparência.
<code>app.config.ts</code>	Providers de âmbito aplicacional (router, HTTP, ...).
<code>app.routes.ts</code>	Definição das rotas (módulo 3).
<code>main.ts</code>	Ponto de entrada: faz o bootstrap do componente raiz.
<code>index.html</code>	Página inicial; o build injeta aqui os bundles.
<code>styles.css</code>	Estilos globais.
<code>angular.json</code>	Configuração do CLI: build, estilos, schematics.
<code>package.json</code>	Dependências e scripts npm.
<code>tsconfig.json</code>	Configuração do compilador TypeScript.

O que já não existe

Sem `app.module.ts`, sem `environments` por defeito, sem `polyfills.ts` explícito, `angular.json` muito mais curto.

Tudo é uma árvore

A aplicação é uma árvore de componentes com `App` no topo. Tudo o resto — routing, DI, HTTP — liga-se a essa árvore.



O componente raiz

```
app.ts

// src/app/app.ts
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('skyboard');

  protected updateTitle(): void {
    this.title.set('Skyboard - flights, simplified');
    console.log('Title updated', this.title());
  }
}
```

```
app.html

<!-- src/app/app.html -->
<h1>Hello, {{ title() }}</h1>
<button (click)="updateTitle()">
  Update Title
</button>
```

- **Standalone por defeito:** não há standalone: true nem NgModule — imports declara o que o template usa.
- **protected readonly:** só o template lhe toca; o signal não é substituído, é atualizado.
- **{{ title() }}**: ler um signal é chamá-lo como função.
- **(click):** event binding — parêntesis para eventos, parêntesis retos para propriedades.



Signal: o primitivo reativo

Um objeto que guarda um valor e avisa quem depende dele quando esse valor muda. O Angular usa isso para atualizar só a UI que realmente mudou.

API essencial

```
const title = signal('skyboard');    // WritableSignal<string>

title();                             // ler → 'skyboard'
title.set('Skyboard 2');             // escrever
title.update(t => t.toUpperCase());  // projetar o valor atual

const count = signal(0);
count.update(c => c + 1);

const readonlyCount = count.asReadonly(); // Signal<number>
```

Sempre com valor

Um signal nunca está 'vazio': recebe um valor inicial e o tipo é inferido — ou explícito: `signal<Flight[]>([])`.

Signal vs WritableSignal

`Signal<T>` só lê. `WritableSignal<T>` lê, set e update. Exponha read-only para fora com `asReadonly()`.

Na sessão 5 entramos a fundo: computed, effect, linkedSignal, untracked e interop com RxJS.



Como a aplicação arranca

main.ts

```
// src/main.ts
import { bootstrapApplication }
  from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

app.config.ts

```
// src/app/app.config.ts
import { ApplicationConfig,
  provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
  ],
};
```

index.html

<app-root>



main.ts

bootstrapApplication



app.config.ts

providers



App

árvore de componentes

Os `provide*()` são o ponto de extensão: HTTP, animações, zoneless — tudo entra aqui. Sem `AppModule`.



Afinar o CLI antes de gerar código

```
angular.json
// angular.json → projects/skyboard/schematics
"schematics": {
  "@schematics/angular:component": {
    "style": "none",
    "skipTests": true,
    "changeDetection": "OnPush"
  },
  "@schematics/angular:directive": { "skipTests": true },
  "@schematics/angular:pipe":       { "skipTests": true },
  "@schematics/angular:service":    { "skipTests": true }
}
```

OnPush + signals: o Angular só re-verifica o componente quando um signal ligado ao template muda. É a base do capítulo zoneless (módulo 8).

Menos ruído durante a formação

Sem .css nem .spec por componente: menos ficheiros para navegar e o foco fica no .ts e no .html.

OnPush: explícito ou implícito?

Desde o Angular 22, OnPush é o default — o CLI já não escreve a linha. Escrevê-la torna a estratégia visível e o snippet portátil para projetos em versões anteriores. Escolham uma regra e sejam consistentes.



Qualidade e build

ng lint

Primeira execução configura o ESLint (aceitar defaults). Regras em `eslint.config.js`.

Com a extensão ESLint no VS Code os erros aparecem enquanto escrevem — o code review fica livre para discutir arquitetura, não imports por usar.

ng build

Compila TS → JS, faz bundling, minificação e tree-shaking.

Resultado em `dist/skyboard/browser` — ficheiros estáticos, deploy = copiar para um servidor web.

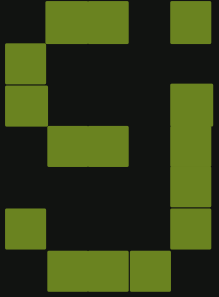
`ng serve` usa o mesmo pipeline sem otimizações e com source maps.

npm 12 e install scripts

Desde o npm 12 os scripts de instalação de dependências (`esbuild`, `@parcel/watcher`, ...) são bloqueados por defeito.

O starter do Skyboard já traz `allowScripts` no `package.json` — o npm install corre limpo.

Nos vossos projetos: `npm install-scripts ls / approve <pkg>`.



PARTE 2

Componentes baseados em Signals

Modelo · componente · template · control flow · HTTP · sub-componentes



Gerar o primeiro componente da feature

```
terminal
ng generate component flight-search
ng g c flight-search      # forma curta

# resultado (com a config anterior):
src/app/flight-search/
├─ flight-search.ts
└─ flight-search.html
```

```
flight-search.ts (gerado)

import { ChangeDetectionStrategy, Component }
  from '@angular/core';

@Component({
  selector: 'app-flight-search',
  imports: [],
  templateUrl: './flight-search.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class FlightSearch {}
```

Style guide atualizado: sem sufixos

Já não é FlightSearchComponent / flight-search.component.ts. O CLI gera FlightSearch / flight-search.ts. Podem usar sufixos próprios com mais significado (...Search, ...Edit, ...Card).

Selector e prefixo

app-flight-search → <app-flight-search /> (self-closing permitido). O prefixo app evita colisões com elementos nativos e libs; muda-se em angular.json e eslint.config.js (chave prefix).



O modelo de dados

```
flight.ts

// flight-search/data/flight.ts
import { Aircraft, initialAircraft } from './aircraft';
import { Price } from './price';

export interface Flight {
  id: number;
  from: string;
  to: string;
  date: string;          // ISO: 2030-12-24T17:00+01:00
  delayed: boolean;
  delay: number;
  aircraft: Aircraft;
  prices: Price[];
}

export const initialFlight: Flight = {
  id: 0, from: '', to: '', date: '',
  delayed: false, delay: 0,
  aircraft: initialAircraft, prices: [],
};
```

```
aircraft.ts · price.ts

// aircraft.ts
export interface Aircraft {
  type: string;
  registration: string;
}

export const initialAircraft: Aircraft = {
  type: '', registration: '',
};

// price.ts
export interface Price {
  flightClass: string;
  amount: number;
}
```

Porquê initialFlight?

Um valor por defeito completo evita null/undefined a espalhar-se pela app e dá um ponto de partida seguro para `signal<Flight>(initialFlight)`.



Estado do componente com Signals

flight-search.ts

```
import { ChangeDetectionStrategy, Component, signal } from '@angular/core';
import { form } from '@angular/forms/signals';
import { Flight } from '../data/flight';

@Component({ /* ... */ })
export class FlightSearch {
  protected readonly filter = signal({ from: 'Lisbon', to: 'Porto' });
  protected readonly filterForm = form(this.filter);

  protected readonly flights = signal<Flight[]>([]);
  protected readonly selectedFlight = signal<Flight | null>(null);

  protected search(): void {
    const date = new Date().toISOString();
    this.flights.set([
      { id: 1, from: this.filter().from, to: this.filter().to, date,
        delayed: false, delay: 0, aircraft: { ...initialAircraft }, prices: [] },
      // ... mais dois voos estáticos
    ]);
  }

  protected select(f: Flight): void {
    this.selectedFlight.set(f);
  }
}
```

- **Tudo o que o template usa** são propriedades e métodos protected.
- **signal<Flight[]>([])** — o tipo não se infere de um array vazio: explicitar.
- **Flight | null** para 'nada selecionado' — sem undefined implícito.
- **form(this.filter)** liga o signal a um formulário (Signal Forms). Detalhe no módulo 7.
- **Dados estáticos primeiro** — testar template e UI antes de HTTP.



Template e data binding

```
flight-search.html

<h1>Flight Search</h1>
<form>
  <label for="from">From:</label>
  <input [formField]="filterForm.from" id="from" />
  <label for="to">To:</label>
  <input [formField]="filterForm.to" id="to" />
  <button type="button" (click)="search()"
    [disabled]="!filter().from || !filter().to">
    Search
  </button>
</form>

@if (flights().length > 0) {
  <table>
    @for (flight of flights(); track flight.id) {
      <tr [class.selected]="selectedFlight() === flight"
        [style.color]="flight.delayed ? 'darkred' : 'black'"
        (click)="select(flight)" class="selectable">
        <td>{{ flight.id }}</td>
        <td>{{ flight.from }} → {{ flight.to }}</td>
        <td>{{ flight.date | date }}</td>
      </tr>
    }
  </table>
}
<pre>{{ selectedFlight() | json }}</pre>
```

(click)**Event binding**

DOM → componente. Igual para (change), (keydown), ...

[disabled]**Property binding**

componente → propriedade DOM. [value], [checked], [src], ...

{{ }}**Interpolação**

mostrar valores; os signals lêem-se com ().

[class.x] / [style.y] **Styling condicional**

classes e estilos inline a partir de expressões.

| date · | json**Pipes**

transformar/formatar valores no template.

type="button" é obrigatório: o default é submit e recarregaria a página inteira.



Control flow nativo: @if, @for, @switch

@for com track e @empty

```
@for (flight of flights(); track flight.id) {
  <tr>...</tr>
} @empty {
  <tr><td colspan="5">No flights found.</td></tr>
}
```

@switch — vários valores por @case (v21.1+)

```
@switch (passenger().status) {
  @case ('SEN') @case ('A') { <p>Senator</p> }
  @case ('FQT') @case ('B') { <p>Frequent Traveller</p> }
  @default { <p>Regular Passenger</p> }
}
```

@switch exaustivo (v21.2 / v22)

```
@switch (loyalty().kind) {
  @case ('senator') { ... }
  @case ('frequent') { ... }
  @case ('regular') { ... }
  @default never(loyalty()); // ou: @default never;
}
```

- **Sintaxe @** segue o JavaScript; nada para importar, nenhuma diretiva no bundle.
- **track é obrigatório** — identificador estável (id). Sem ele, o Angular recriaria o DOM inteiro; com ele, move elementos existentes. Fallbacks: track \$index, track flight.
- **@empty** resolve o velho padrão *ngIf + *ngFor para listas vazias.
- **@default never** pede ao compilador a verificação de exaustividade: adicionar um caso ao union sem @case parte o build. never(expr) para discriminated unions.



Pipes e ligar a feature ao shell

pipes com parâmetros

```
<td>{{ flight.date | date:'dd.MM.yyyy HH:mm' }}</td>
<td>{{ price.amount | number:'1.2' }}</td>
<pre>{{ selectedFlight() | json }}</pre>
```

- **Importar no componente:** DatePipe, DecimalPipe, JsonPipe de @angular/common no array imports.
- **date** aceita Date ou ISO string; number formata por locale.
- **json** é o vosso amigo para debugging de signals no template.

Regra

Se está no template, tem de estar em imports — componente, diretiva ou pipe. O compilador avisa.

app.ts

```
// src/app/app.ts
import { FlightSearch } from '../flight-search/flight-search';

@Component({
  selector: 'app-root',
  imports: [Navbar, Sidebar, FlightSearch],
  templateUrl: './app.html',
})
export class App {}
```

app.html

```
<!-- src/app/app.html -->
<div class="wrapper">
  <div class="sidebar"><app-sidebar /></div>
  <div class="main-panel">
    <app-navbar />
    <div class="content">
      <app-flight-search />
    </div>
  </div>
</div>
```



Quando não funciona: debugging

1 Consola do browser

Erros do Angular trazem links diretos para a linha do .html/.ts. Ler a mensagem inteira antes de pesquisar.

2 Debugger do Chrome

ng serve gera source maps → breakpoints em TypeScript no separador Sources. Ctrl/Cmd+Shift+P para saltar para um ficheiro.

3 VS Code (F5)

Breakpoint na margem do .ts, Run › Start Debugging. O CLI já gera .vscode/launch.json — zero configuração.

4 Angular DevTools

Extensão do browser: árvore de componentes, signals, profiler de change detection. Voltamos a ela no módulo 8.